Autonomous Perception & Planning for ADAS — YOLO + BEV on nuScenes

A two-stage driving-perception stack that detects road obstacles from camera images and predicts where the vehicle should go next from LiDAR — packaged as a live demo you can run in one command.

This project takes raw sensor data from the nuScenes autonomous-driving dataset and turns it into the two signals an Advanced Driver-Assistance System (ADAS) needs to avoid a collision: *what is around the car* (object detection) and *where the car is about to be* (ego-trajectory prediction). Both models are trained, both run end-to-end, and both are wired into an interactive Streamlit app.

The business problem

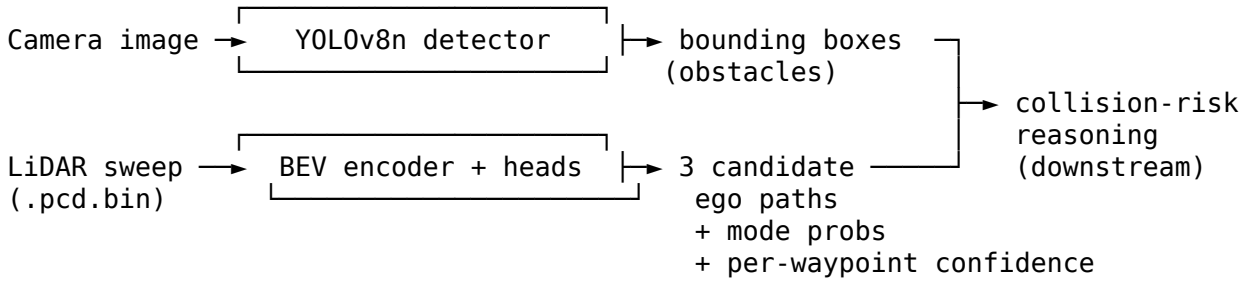
Forward-collision warning and automatic emergency braking are now mandated or incentivised in most major markets, and the underlying capability — *perceive the scene, predict the path, flag the conflict* — is the core of every driver-assistance and autonomy product. Building it well is expensive because it spans two hard sub-problems that are usually solved by separate teams.

This project demonstrates a single engineer owning that full slice:

- **Perception** — detect vehicles, pedestrians and other obstacles in the camera feed so the system knows what to react to.
- **Planning / prediction** — from a top-down LiDAR view, predict the most likely short-horizon path of the ego vehicle, with several alternative paths and a confidence on each, so a safety layer can reason about whether the planned motion intersects a detected obstacle.

The value is concrete: a fused detect-and-predict module is the input to collision-risk scoring for ADAS, to driver-risk telematics for fleet and insurance products, and to the perception layer of a research AV stack. The same code path serves all three; only the consumer of the output changes.

System architecture



Two complementary sensors, two models, one decision surface. The detector answers *what and where are the obstacles*; the planner answers *where am I going*. Overlaying the two in the same bird's-eye coordinate frame is what lets a safety layer decide whether to warn or brake.

Model 1 — YOLOv8n obstacle detector

A YOLOv8-nano detector fine-tuned on **20,256** nuScenes camera frames. It runs on a single object class (obstacle), which keeps the warning logic simple: anything detected with sufficient confidence is a thing to avoid.

For cases where the *type* of object matters (car vs. truck vs. pedestrian), the demo also offers a stock **COCO-pretrained YOLOv8** filtered to road classes (`person, bicycle, car, motorcycle, bus,`

`truck`) — named detection with no extra training. The two detectors are selectable side-by-side in the app.

Model 2 — BEV multi-modal trajectory planner

A custom convolutional network that consumes a **500×500×3 bird's-eye-view** raster built from the LiDAR point cloud (height / intensity / density channels) and outputs:

- **3 candidate trajectories**, each 20 future waypoints in the ego frame (≈ the next several seconds of motion);
- a **probability over the 3 modes** (which path the model trusts most);
- a **per-waypoint confidence**;
- an auxiliary **8-class BEV semantic segmentation** head used during training to force the encoder to learn road structure.

Multi-modality matters for safety: at an intersection the "correct" future is not unique, so collapsing to a single predicted path hides risk. Predicting several weighted hypotheses is the standard formulation in modern motion-forecasting (and the right one for a risk layer that must consider the worst plausible case).

The architecture: a VGG-style encoder (stem → three down-sampling double-conv stages, 64→128→256→512 channels) feeds a 256-d scene embedding shared by four heads (three trajectory regressors, a mode classifier, a confidence head, and a transposed-conv segmentation decoder). Roughly 12M parameters.

Results

All numbers below are read directly from the training logs in this repo (`runs/train/nuscenes_yolo/results.csv` and the BEV checkpoint metadata) — nothing is invented.

Detection (YOLOv8n, 20 epochs, CPU, single class)

Metric	Value
Precision	0.64
Recall	0.30
mAP@0.50	0.35
mAP@0.50–0.95	0.21
Training images	20,256
Training time	~68 min (CPU)

Real detections on held-out validation frames (blue boxes are model predictions with confidence scores):

The metrics climb monotonically across all 20 epochs and had **not** plateaued — mAP@0.5 rose from 0.04 → 0.35 — so the detector is clearly *undertrained* rather than converged. It was trained on CPU as a single class for speed; the limiting factor was compute, not the pipeline. The precision/recall and confusion curves tell the same story:

Trajectory prediction (BEV planner)

Metric	Value
Validation ADE (avg displacement error)	3.29 m
Validation FDE (final displacement error)	7.23 m

Metric	Value
Checkpoint epoch	5
Trajectory modes	3
Prediction horizon	20 waypoints

A live prediction: the three coloured paths are the candidate modes (alpha-weighted by probability), the red path is the selected mode, and the star is the ego vehicle. The right panel is the per-waypoint confidence.

This is an **early checkpoint (epoch 5)** and it behaves like one: it has learned a sensible "continue forward" prior with ~3.3 m average error, but it has not yet learned to commit to turns — the three modes have largely collapsed toward the same straight path, and the auxiliary segmentation head is still near-empty. That is exactly the failure mode the v3 training script (`train_trajectory_v3.py`) was written to fix, with rotation/flip augmentation and a mode-diversity loss to break the straight-line collapse. The architecture and data pipeline are sound; the checkpoint simply needs more epochs.

I'm calling this out deliberately: the point of the project is a correct, runnable, well-engineered stack, not a leaderboard number.

From inference to a product: collision-risk fusion

A detector that draws boxes and a planner that draws a path are tech demos. The *product* is the decision you get when you fuse them over a continuous drive. That fusion is the actual ADAS deliverable, and it's implemented in `collision_risk.py`:

> For every frame, take the planner's predicted ego path and walk along it through a vehicle-width corridor. If the LiDAR bird's-eye-view shows an **elevated** obstacle (height above the road surface, so curbs and ground are ignored) inside that corridor — beyond the ego vehicle's own footprint — report the distance to it, an estimated time-to-obstacle, and a **CLEAR / CAUTION / BRAKE** decision.

This runs as two operator-facing tools, both over real driving sequences rather than one frame at a time.

1. Live collision-risk monitor (`risk_monitor.py`)

Processes a continuous LiDAR scene and renders an annotated video with a live risk banner — the view an ADAS validation engineer or a fleet operator would actually watch. Ego in the centre, predicted path in blue, in-path obstacles highlighted, and the decision up top.

2. Fleet safety report (`safety_report.py`)

Runs an entire scene and produces a one-page PDF: a 0–100 scene safety score, how much of the drive was spent at each risk level, the risk/distance/speed timeline, and a ranked list of discrete risk events with their closest approach and time-to-obstacle. This is the artefact an insurance-telematics or fleet-safety team reads — it turns raw sensor logs into a quantified risk profile per drive.

Why this is the business value. The same fused pipeline serves three markets without changing the core code, only the consumer of the output: a real-time *warn/brake* signal for ADAS, a per-drive *risk score* for fleet and usage-based-insurance products, and a *scene risk profile* for AV validation and triage. Distance-to-obstacle and time-to-obstacle are the quantities every one of those products is ultimately built on.

(The risk logic is validated on the bundled real LiDAR frames; across a sample 120-frame scene it produces a realistic spread of CLEAR/CAUTION/BRAKE decisions with closest approaches of 3.5–33 m. With an early trajectory checkpoint the planner's path is mostly straight, so the monitor can also



Figure 1: YOLO detections

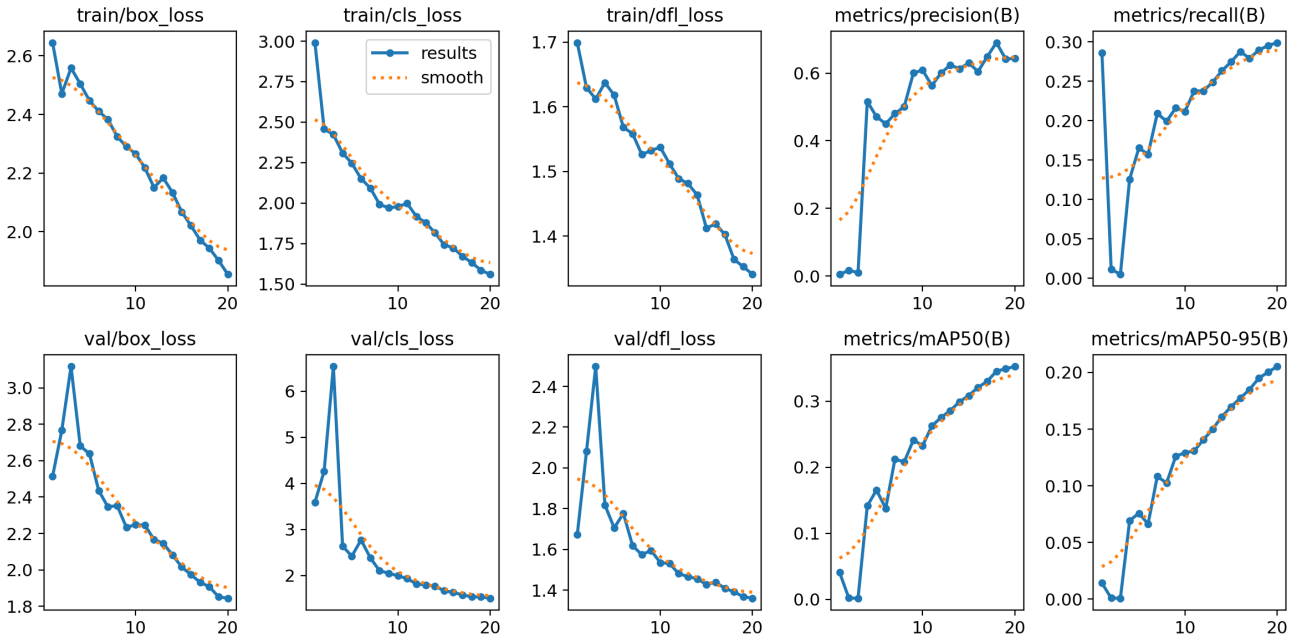


Figure 2: YOLO training curves

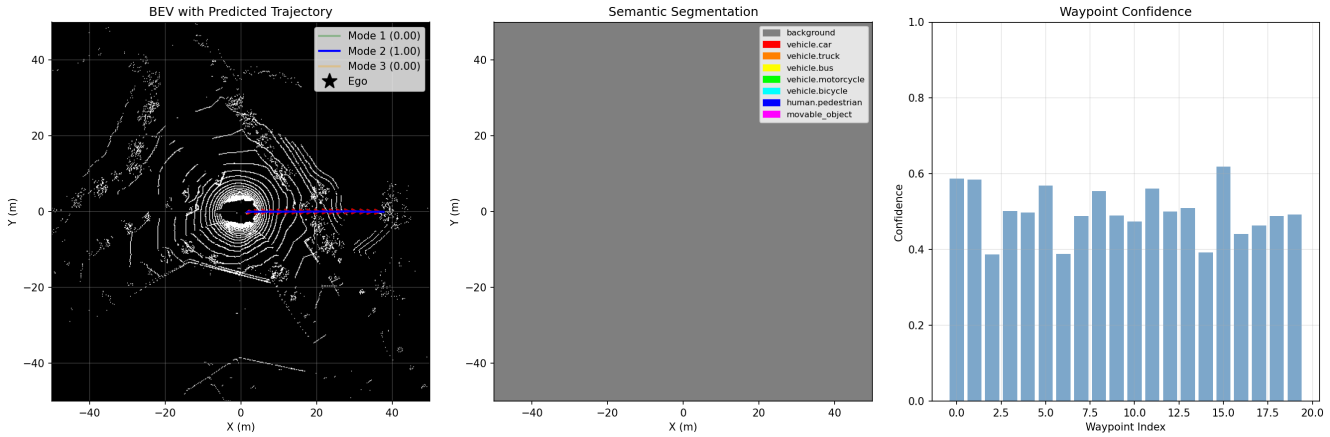


Figure 3: BEV trajectory prediction

run on a constant-velocity baseline path — `--straight` — to exercise the fusion independently of the planner.)

What this repo actually delivers

The original project was developed on AWS SageMaker against the full ~200 GB nuScenes dataset. This package strips that down to something a reviewer can run on a laptop in minutes:

- **`trajectory_model.py`** — the BEV planner architecture. (The original definition was lost with the SageMaker instance; it was reconstructed from the trained checkpoint and verified to match all 130 weight tensors exactly, so the weights load with `strict=True`.)
- **`collision_risk.py`** — the perception+planning fusion core (path × obstacle map → CLEAR/CAUTION/BRAKE). Pure NumPy, unit-testable.
- **`risk_monitor.py`** — renders the annotated collision-risk video over a driving sequence.
- **`safety_report.py`** — generates the per-scene PDF safety report.
- **`run_inference.py`** — one-command inference (`yolo` / `named` / `bev` / `all`) on the bundled sample data.
- **`scene_dashboard.py`** — composes the camera + BEV side-by-side dashboard and encodes a sequence to GIF/MP4.
- **`app.py`** — a Streamlit demo: live detection (fine-tuned *or* named COCO), live trajectory + risk on a single frame, and a **multi-frame scene dashboard that renders an animated GIF** (camera + BEV + risk) over a chosen sequence.
- **`data_preparation_v2.py` / `train_trajectory_v2.py` / `train_trajectory_v3.py`** — the original data pipeline and training scripts.
- Bundled sample data: **3,376 LiDAR frames** + the YOLO image set, plus the trained weights — so no nuScenes download is required to see it work.

Run it

```
# 1. install (CPU is fine for inference)
pip install -r requirements.txt

# 2. interactive demo — runs both models live in the browser
streamlit run app.py

# 3. or batch inference straight to ./inference_outputs/
python run_inference.py named     # NAMED detection (car/truck/bus/person/…)
python run_inference.py yolo       # fine-tuned single-class detector
```

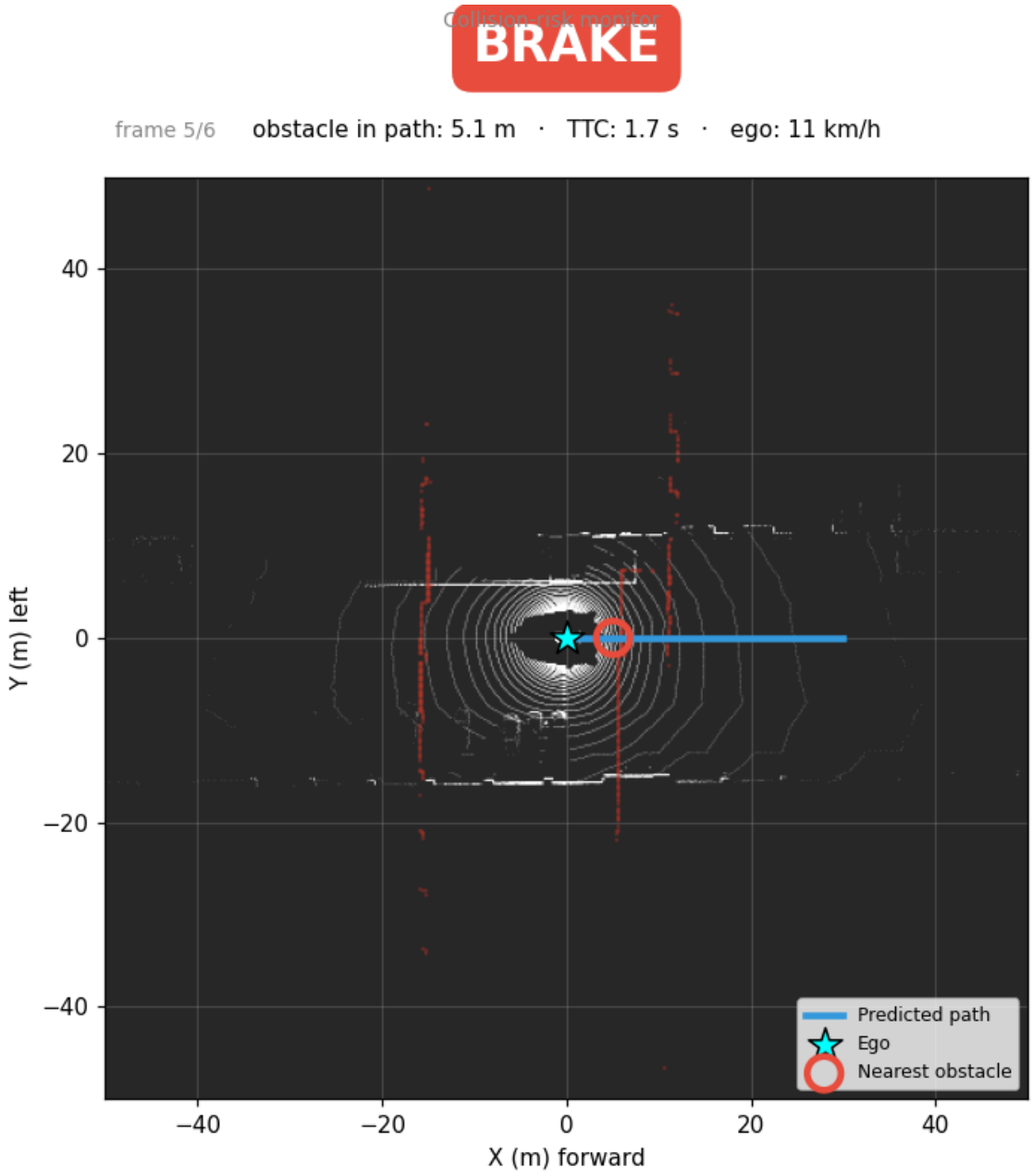


Figure 4: Risk monitor frame

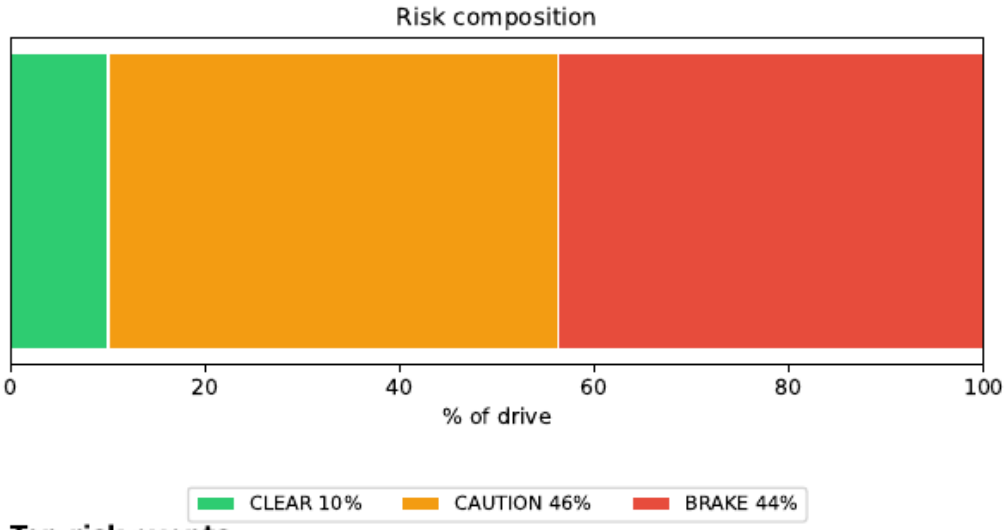
Drive Safety Report

Scene safety score: 3/100

```
Scene             : n008-2018-08-01-15-16-36-0400
Frames analysed   : 80  (~40 s at 2 Hz key-frames)
Path source       : constant-velocity baseline

Time CLEAR        : 10%   (8 frames)
Time CAUTION      : 46%   (37 frames)
Time BRAKE        : 44%   (35 frames)

Risk events       : 4   (of which 4 reached BRAKE)
Closest approach  : 3.7 m
```



Top risk events

frame range	dur	peak	min dist	min TTC
0–56	28.5s	BRAKE	3.7 m	1.2s
58–58	0.5s	BRAKE	3.7 m	1.2s
64–76	6.5s	BRAKE	4.7 m	1.6s
79–79	0.5s	BRAKE	6.4 m	2.1s

Generated by safety_report.py · perception + planning + collision-risk fusion on nuScenes LiDAR

Figure 5: Safety report

```
python run_inference.py bev        # trajectory prediction on LiDAR frames

# 4. the business-value tools — collision risk over a whole driving scene
python risk_monitor.py      # -> inference_outputs/risk_monitor.mp4
python safety_report.py     # -> inference_outputs/safety_report.pdf
#   (add --straight to run the fusion on a baseline path, no planner)

# 5. sanity-check that the reconstructed model loads the weights
python trajectory_model.py
```

Engineering skills demonstrated

- **End-to-end ownership** of a multi-sensor (camera + LiDAR) perception/planning stack, from raw `.pcd.bin` point clouds to a deployed demo.
- **Custom multi-task model design** in PyTorch: a shared encoder with four heads (multi-modal regression + classification + confidence + segmentation).
- **Motion-forecasting fundamentals**: multi-modal trajectory prediction with ADE/FDE evaluation, the standard metrics in the field.
- **Cloud training** on AWS SageMaker against a 200 GB dataset.
- **Productionisation**: Streamlit app, one-command inference, portable relative paths, pinned dependencies.
- **Turning models into a product**: fused perception + planning into a collision-risk decision (distance/TTC, CLEAR/CAUTION/BRAKE) and shipped it as both a live monitor video and a quantified per-scene safety report.
- **Debugging real failure modes**: diagnosed straight-line mode collapse and wrote the augmentation + diversity-loss fix.
- **Reverse-engineering a model from its weights** to recover a lost architecture — recovering all 130 parameter tensors exactly.

What I'd do next

1. Resume BEV training past epoch 5 with the v3 augmentation/diversity-loss recipe to break the straight-line collapse and bring FDE down — which directly sharpens the risk monitor's path.
2. Train YOLO on GPU for more epochs with multiple object classes (the curves show it was still improving at epoch 20).
3. Calibrate the risk thresholds and TTC against labelled near-miss data, and project the camera detections into the BEV frame